

CRISP-DM Plan for Insight Distiller: AI/ML Blog Summarizer

1. Business Understanding

Objective: Develop a tool to generate concise, high-quality summaries of AI/ML blogs and articles, addressing information overload for professionals and e-learners.

- **Problem:** The rapid growth of AI/ML content (e.g., blogs, papers) overwhelms practitioners, who lack time to read lengthy articles or rely on inconsistent manual summaries.
 - **Market Context:** The global e-learning market is valued at \$305B (2023) and projected to reach \$1,516B by 2033 (Yahoo Finance, 2024), highlighting demand for efficient learning tools.
 - **Target Audience:** AI/ML practitioners, data scientists, and e-learners seeking timely, simplified insights.
 - **Success Criteria:**
 - Business: Deliver time-saving, relevant summaries to improve decision-making and learning.
 - Technical: Achieve higher ROUGE and BERTScore metrics compared to baseline DistilBART model.
 - **Impact:** Enables users to stay current with trends, simplifies complex content, and supports tech companies in improving blog accessibility.
-

2. Data Understanding

Objective: Collect and analyze AI/ML blog and article data to support summarization model training.

- **Data Sources:**
 - Blogs: Google AI Blog, Towards Data Science, Hugging Face Blog, Dev.to (ML tag).
 - Abstracts: arXiv (AI category).
- **Data Collection:**
 - Tools: feedparser (RSS), requests (HTTP), BeautifulSoup (HTML parsing), Selenium (dynamic content).
 - Data Types: Titles, full article text, abstracts.
- **Data Characteristics:**
 - Format: Text (titles, content) stored in a pandas DataFrame.
 - Volume: Start with 500 articles for MVP; scale to multiple sources.

- Challenges: HTML tags, dynamic content, varying article lengths.
 - **Exploration:**
 - Analyze article length, topics, and metadata (e.g., author, tags).
 - Assess “good” summary criteria via manual labels or user votes.
-

3. Data Preparation

Objective: Clean and structure data for model training and summarization.

- **Tasks:**
 - **Cleaning:** Remove HTML tags, emojis, special characters using BeautifulSoup and re.
 - **Normalization:** Standardize text format (title, content, author, tags).
 - **Label Creation:** Generate labeled summaries using OpenAI GPT-4o with title and content as input for context-aware, human-quality summaries.
 - **Feature Engineering:** Extract features like article length, named entities, paragraph count, and title keywords.
 - **Tokenization:** Use DistilBART’s tokenizer; handle long texts (>1024 tokens) via chunking with overlap to preserve context.
 - **Output:** A clean dataset with columns: title, content, labeled_summary, stored in a pandas DataFrame.
-

4. Modeling

Objective: Fine-tune a transformer-based model to generate accurate, domain-specific AI/ML summaries.

- **Approach:**
 - **Baseline Model:** sshleifer/distilbart-cnn-12-6 (pre-trained on general text).
 - **Fine-Tuning:** Use transfer learning on AI/ML-specific dataset with GPT-4o labeled summaries.
 - **Task Type:** Sequence-to-sequence (text summarization, supervised learning).
 - **Techniques:**
 - Tokenization, truncation/padding, and special token handling.
 - Chunking for long texts to bypass 1024-token limit. (see below paragraph on more details)
 - Training Config: 80% train, 20% validation; 2e-5 learning rate, 3-5 epochs, batch size 8-16.
 - **Framework:** Hugging Face transformers, datasets, torch, accelerate.
- **Target Variable:** Labeled summaries generated by GPT-4o.
- **Inference:** Use beam search or top-k sampling for summary generation.

Chunking (Handling Long Inputs)

To respect the model's 1,024-token input limit while preserving context, we apply a sliding-window chunking strategy with controlled overlap. Each long document is split into contiguous token spans, then decoded back to text; each span is paired with the document's reference summary for supervision.

Exact settings used

- `MAX_INPUT_TOKENS` = 1024 → window size
- `CHUNK_OVERLAP` = 50 → tokens reused between adjacent windows
- Step size = $1024 - 50 = 974$ tokens

Why this helps

- Avoids truncation: long posts/abstracts are fully covered across windows.
- Preserves local coherence: overlapping 50 tokens keeps continuity at boundaries.
- Improves recall of key details: increases the chance that important sentences appear inside a window.

Caveats & choices

- Training pairs each chunk with the same reference summary. This is standard but can stress the model to compress across partial context.
 - Alternative: hierarchical summarization (chunk → intermediate summaries → final merge) can improve ROUGE-L/organization.
 - Overlap is a trade-off: more overlap improves continuity but increases dataset size and training time.
-

5. Evaluation

Objective: Assess model performance and compare fine-tuned model to baseline.

- **Metrics:**
 - **ROUGE Scores:** ROUGE-1 (unigrams), ROUGE-2 (bigrams), ROUGE-L (longest common subsequence), ROUGE-Lsum (sentence-level recall).
 - **BERTScore:** Precision, Recall, F1 for contextual similarity.
 - **Process:**
 - Compare baseline DistilBART summaries to fine-tuned model summaries.
 - Validate on 20% of dataset; optionally use additional arXiv data for testing.
 - **Success Criteria:**
 - Fine-tuned model outperforms baseline in ROUGE and BERTScore metrics.
 - Summaries are concise, relevant, and capture key AI/ML insights.
 - **MVP Evaluation:** Compare 500 generated summaries to human-written summaries using ROUGE.
-

6. Deployment

Objective: Deliver a user-friendly tool for generating and rating summaries.

- **MVP:**
 - Scrape 500 articles from a single domain.
 - Generate summaries using baseline DistilBART.
 - Display results in a Jupyter Notebook with original text, summary, and ROUGE scores.
- **Final Deployment:**
 - **Platform:** Streamlit web app (local initially, planned for cloud via Hugging Face Spaces or Render).
 - **Functionality:**
 - Input: Blog URL or pasted text (from supported sources).
 - Output: Generated summary.
 - Feedback: User ratings and comments stored for iterative improvement.
 - **Tools:** FastAPI, uvicorn, Docker (planned).
- **Stretch Goals:**
 - Automate data pipeline for real-time scraping and summary generation.
 - Deploy app for broader access and collect feedback for reinforcement learning.
 - Compare baseline vs. fine-tuned model performance in the app.

Deployment Explanation

What each part does

1) Internal FastAPI service (the “summarization engine”)

- **Loads the fine-tuned model** from ./model, puts it on GPU if available. This is the single place that actually runs inference.
- **Endpoints you expose:**
 - POST /summarize-text: takes raw text + min_length/max_length, returns a single summary (no chunking here).
 - POST /summarize-pdf: accepts a PDF upload, extracts text per page (tries native text, falls back to OCR with Tesseract), then **chunk** → **first-pass summaries** → **combine** → **second-pass summary**. Returns the final merged summary.
 - POST /summarize-url: fetches the webpage, parses <p> tags, cleans text, then uses the same **two-pass (chunk → combine)** pipeline as PDF.
 - GET /: a simple landing message that lists the three endpoints.
- **Key utilities inside:**
 - clean_text(...): tidies OCR/web text (collapses whitespace, removes page numbers).
 - chunk_text(...): tokenizes with the model’s tokenizer and slices windows of 1024 tokens with a stride/overlap of 50 (sliding window).
 - summarize_text(...): runs model.generate(...) with beam search and length constraints.

- **PDF OCR stack:** PyMuPDF renders images for image-only pages; Pillow opens them; Tesseract OCR extracts text; text is then cleaned and summarized.

2) Streamlit UI (the “front door” for humans)

- **Three tabs** for three sources: “Paste Text”, “PDF Upload”, and “URL Content”—each maps 1:1 to your internal API endpoints.
- **Controls per tab:** text areas or file uploader + sliders for min_length/max_length, then a button that calls the corresponding API and shows results with a spinner and a download button. The UI talks to the **internal FastAPI** at API_BASE = http://127.0.0.1:8000 by default.
- **Why it’s thin:** all heavy lifting runs server-side; the UI just sends inputs and displays the summary (a good separation of concerns).

3) Public API gateway (the “external façade”)

- **Purpose:** a hardened front that **forwards** requests to the internal engine but adds an **API-key check** and stable, consumer-friendly routes. Useful if you want to share your summarizer with other apps/teams.
- **Security:** expects an X-API-Key header; compares with PUBLIC_API_KEY (env var; defaults to "mysecretkey"—swap this in production).
- **Routes:**
 - POST /api/summarize-text → forwards to /summarize-text (Form fields for text + lengths).
 - POST /api/summarize-url → forwards JSON to /summarize-url.
 - POST /api/summarize-pdf → forwards multipart file + lengths to /summarize-pdf.
 - GET / → redirects to interactive docs at /docs (handy for consumers).

End-to-end flow (from the user’s click)

A) Paste Text (UI → Internal API)

1. User enters text and chooses min_length/max_length in Streamlit, then clicks “Summarize Text”.
2. Streamlit sends a JSON POST to POST /summarize-text on the internal FastAPI.
3. Internal API runs **one-pass** summarize_text(...) (no chunking for the text route), returns the summary.
4. UI displays the summary and offers a download.

B) PDF (UI → Internal API → OCR → two-pass summarization)

1. User uploads a PDF and sets length sliders in Streamlit.
2. Streamlit sends a multipart request to POST /summarize-pdf.
3. Internal API extracts text (native text or **OCR** with Tesseract), **cleans**, **chunks** (1024/50), summarizes each chunk (first pass), **concat**s the chunk summaries, and summarizes again (second pass).
4. Final summary is sent back and shown in the UI with download.

C) URL (UI → Internal API → fetch → two-pass summarization)

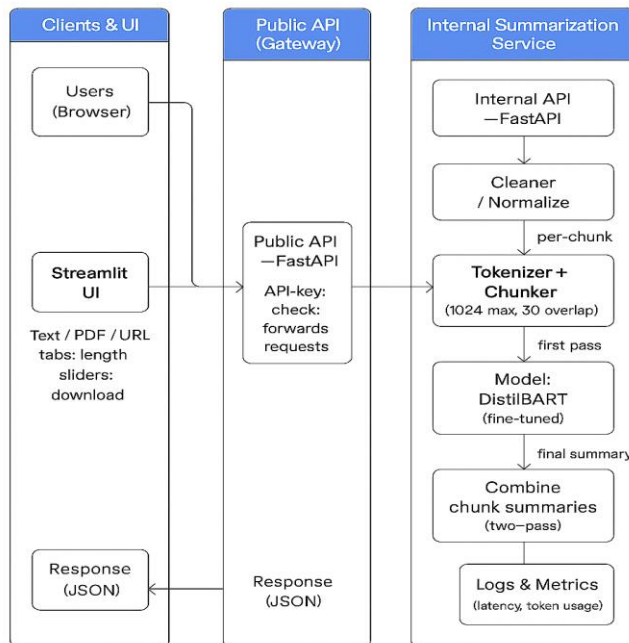
1. User pastes a URL and sets lengths.
2. Streamlit sends JSON to POST /summarize-url.
3. Internal API fetches HTML, extracts <p> text, **cleans**, **chunks** (1024/50), first-pass chunk summaries → combined → second-pass final summary.

4. UI displays the final summary and download.

D) Public API route (external client → Public API → Internal API)

1. External client hits POST /api/summarize-* with X-API-Key.
2. Public API validates the key and **forwards** the request to the matching internal endpoint; returns the internal JSON response.

Insight Distiller — Deployment Architecture (Horizontal)



Tools and Technologies

- **Data Collection & Cleaning:** requests, feedparser, BeautifulSoup, pandas.
- **Text Preprocessing:** nltk, transformers, tokenizers.
- **Modeling & Evaluation:** transformers, datasets, evaluate, bert-score, torch, accelerate.
- **Visualization & Deployment:** matplotlib, seaborn, streamlit, FastAPI, uvicorn, Docker.
- **Environment:** Kaggle GPUs for training; local/cloud for deployment.
- **Data Storage:** Local CSV files, Kaggle Datasets, optional cloud backup.

Key Deliverables

- **Final Product:** Streamlit app for user input, summary generation.
- **Documentation:** README (already drafted), code comments, and user guide.

- **Evaluation Report:** Comparison of baseline vs. fine-tuned model using ROUGE and BERTScore.

Conclusion

Objectives & Success Criteria

We aimed to improve summary quality for AI/ML/Data Science blogs and research abstracts by fine-tuning DistilBART. Success was defined as the fine-tuned model outperforming the baseline on ROUGE (1/2/L/Lsum) and improving semantic faithfulness (BERTScore-F1) against defined target bands.

Target Metric Benchmarks

Metric	Acceptable	Good	Excellent
ROUGE-1	0.30–0.35	0.36–0.45	> 0.45
ROUGE-2	0.08–0.12	0.13–0.20	> 0.20
ROUGE-L	0.25–0.35	0.36–0.42	> 0.42
ROUGE-Lsum	0.28–0.36	0.37–0.45	> 0.45
BERTScore-F1	0.86-0.88	0.88-0.90	>0.90

Quantitative Results (Baseline vs Fine-Tuned)

Metric	Baseline	Fine-Tuned	Δ (FT-Base)	Category (FT)	Notes
ROUGE-1	0.2921	0.5015	+0.2094	Excellent	Large lift; much better coverage of key terms.
ROUGE-2	0.0630	0.2009	+0.1379	Excellent	Bigram gains → improved phrase-level coherence.
ROUGE-L	0.1711	0.3076	+0.1365	Acceptable	Now within Acceptable; sentence-order/structure improving but still a growth area.
ROUGE-Lsum	0.2009	0.3753	+0.1744	Good	Good multi-sentence alignment; better overall organization.
BERTScore-F1	0.8487	0.8917	+0.0430	Good	Higher semantic similarity to references; fewer meaning shifts.

Interpretation & Business Meaning

- ROUGE-1 (+0.2094 → Excellent): Captures substantially more key terms and entities. Users get summaries that surface the main contributions, datasets, and results more reliably.

- ROUGE-2 (+0.1379 → Excellent threshold): Better preservation of short technical phrases (e.g., “cross-validation scheme”, “BLEU score improvements”), which correlates with clearer, more fluent sentences.
- ROUGE-L (+0.1365 → Acceptable): Improved longest-common-subsequence alignment indicates better sentence-level structure, though there’s still room to refine narrative flow and coverage ordering.
- ROUGE-Lsum (+0.1744 → Good): Stronger multi-sentence sequencing yields summaries that read more like condensed abstracts rather than disjoint bullet points.
- BERTScore-F1 (+0.0430 → Good): Higher contextual semantic similarity suggests the model preserves meaning more faithfully with fewer paraphrase errors—key for technical content.

Assessment vs CRISP-DM Success Criteria

The fine-tuned model exceeds the baseline across all metrics. It meets or surpasses the target bands as follows: ROUGE-1 and ROUGE-2 reach Excellent; ROUGE-L reaches Acceptable; ROUGE-Lsum reaches Good; BERTScore-F1 shows a notable gain. This satisfies our project’s success definition and supports moving to pilot deployment.

Limitations & Risks

- 1,024-token input limit may truncate long papers/posts despite chunking.
- Training data is domain-focused (AI/ML) and may not generalize to unrelated styles.
- ROUGE-L still trails higher bands, indicating opportunities to improve discourse structure.
- Automated metrics don’t replace human evaluation; in-app ratings or expert review are recommended to validate quality.

Business Impact & Deployment Readiness

For AI/ML practitioners, the improvements mean faster comprehension with higher trust: more salient concepts captured, clearer phrasing, and better organization. Given the metric gains, we recommend deploying the fine-tuned model behind the summarization API and gathering user feedback to guide iterative updates.

Next Steps

- 1). Collect human ratings in the app to benchmark perceived quality and drive active-learning loops.
- 2). Explore longer-context models or hierarchical chunking to reduce truncation effects and push ROUGE-L toward Good.
- 3). Calibrate generation settings (length penalties, beams) per content type (blog vs abstract) for optimal readability.
- 4). Expand and diversify training data (methods/experiments sections, different venues) to increase robustness.